

Binäre Suche – Integer-Array (Winter 2021/22)

Pseudocode (IHK-Stil):

```
Funktion binarySearch(werte: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(werte) - 1
    while links ≤ rechts
        mitte = (links + rechts) / 2
        if werte[mitte] == x dann
            return mitte
        else if werte[mitte] < x dann
            links = mitte + 1
        else
            rechts = mitte - 1
        end if
    end while
    return -1
end Funktion
```

Java-Beispiel:

```
public static int binarySearch(int[] werte, int x) {
    int links = 0;
    int rechts = werte.length - 1;
    while (links <= rechts) {
        int mitte = (links + rechts) / 2; // ganzzahlig
        if (werte[mitte] == x) {
            return mitte;
        } else if (werte[mitte] < x) {
            links = mitte + 1;
        } else {
            rechts = mitte - 1;
        }
    }
    return -1;
}
```

Erklärung:

Voraussetzung: Array ist aufsteigend sortiert.

Komplexität: $O(\log n)$.

Typische Fehler: Sortierung vergessen; falsche Grenzen; Mitte nicht ganzzahlig.

Binäre Suche – Kurse nach Kursnummer (Winter 2020/21)

Pseudocode (IHK-Stil):

```
Funktion sucheKurs(kurse: Array<Kurs>, nr: Integer) → Kurs
    links = 0
    rechts = länge(kurse) - 1
    while links ≤ rechts
        mitte = (links + rechts) / 2
        if kurse[mitte].getKursnummer() == nr dann
            return kurse[mitte]
        else if kurse[mitte].getKursnummer() < nr dann
            links = mitte + 1
        else
            rechts = mitte - 1
        end if
    end while
    return null
end Funktion
```

Java-Beispiel:

```
public static Kurs sucheKurs(Kurs[] kurse, int nr) {
    int links = 0, rechts = kurse.length - 1;
    while (links <= rechts) {
        int mitte = (links + rechts) / 2;
        int kn = kurse[mitte].getKursnummer();
        if (kn == nr) {
            return kurse[mitte];
        } else if (kn < nr) {
            links = mitte + 1;
        } else {
            rechts = mitte - 1;
        }
    }
    return null;
}
```

Erklärung:

Objekt-Variante: Vergleich erfolgt über den Schlüssel `getKursnummer()`.
Achte darauf, dass `kurse` nach Kursnummer sortiert ist.

Binäre Suche – Bestellungen nach Nummer (Winter 2022/23)

Pseudocode (IHK-Stil):

```
Funktion findeBestellung(bestellungen: Array<Bestellung>, nr: Integer) → Bestellung
    l = 0
    r = länge(bestellungen) - 1
    while l ≤ r
        m = (l + r) / 2
        if bestellungen[m].getBestellnummer() == nr dann
            return bestellungen[m]
        else if bestellungen[m].getBestellnummer() < nr dann
            l = m + 1
        else
            r = m - 1
        end if
    end while
    return null
end Funktion
```

Java-Beispiel:

```
public static Bestellung findeBestellung(Bestellung[] arr, int nr) {
    int l = 0, r = arr.length - 1;
    while (l <= r) {
        int m = (l + r) / 2;
        int b = arr[m].getBestellnummer();
        if (b == nr) return arr[m];
        if (b < nr) l = m + 1;
        else r = m - 1;
    }
    return null;
}
```

Erklärung:

Gleiche Logik wie bei Kursen. Wichtig: nach Bestellnummer sortiert.

Binäre Suche – generisch per Schlüssel-Funktion (IHK-Stil)

Pseudocode (IHK-Stil):

```
Funktion binarySearchByKey(L: Liste<T>, key: Funktion<T → Integer>, such: Integer)
    l = 0
    r = L.size() - 1
    while l ≤ r
        m = (l + r) / 2
        k = key(L.get(m))
        if k == such dann
            return m
        else if k < such dann
            l = m + 1
        else
            r = m - 1
        end if
    end while
    return -1
end Funktion
```

Java-Beispiel:

```
public static <T> int binarySearchByKey(java.util.List<T> list,
                                       java.util.function.ToIntFunction<T> key,
                                       int such) {
    int l = 0, r = list.size() - 1;
    while (l <= r) {
        int m = (l + r) / 2;
        int k = key.applyAsInt(list.get(m));
        if (k == such) return m;
        if (k < such) l = m + 1;
        else r = m - 1;
    }
    return -1;
}
```

Erklärung:

Generisch: key extrahiert den Sortierschlüssel aus T. Sehr prüfungsnah (Function/ToIntFunction).

Binäre Suche – erstes Vorkommen (Duplikate)

Pseudocode (IHK-Stil):

```
Funktion binarySearchErstes(werte: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(werte) - 1
    ergebnis = -1
    while links ≤ rechts
        mitte = (links + rechts) / 2
        if werte[mitte] == x dann
            ergebnis = mitte
            rechts = mitte - 1
        else if werte[mitte] < x dann
            links = mitte + 1
        else
            rechts = mitte - 1
        end if
    end while
    return ergebnis
end Funktion
```

Java-Beispiel:

```
public static int binarySearchErstes(int[] a, int x) {
    int l = 0, r = a.length - 1, res = -1;
    while (l <= r) {
        int m = (l + r) / 2;
        if (a[m] == x) { res = m; r = m - 1; }
        else if (a[m] < x) { l = m + 1; }
        else { r = m - 1; }
    }
    return res;
}
```

Erklärung:

Bei Gleichheit nicht sofort return, sondern Ergebnis merken und links weiter suchen.

Binäre Suche – Einfügeposition (lowerBound)

Pseudocode (IHK-Stil):

```
Funktion lowerBound(werte: Array<Integer>, x: Integer) → Integer
    links = 0
    rechts = länge(werte)
    while links < rechts
        mitte = (links + rechts) / 2
        if werte[mitte] < x dann
            links = mitte + 1
        else
            rechts = mitte
        end if
    end while
    return links
end Funktion
```

Java-Beispiel:

```
public static int lowerBound(int[] a, int x) {
    int l = 0, r = a.length;
    while (l < r) {
        int m = (l + r) / 2;
        if (a[m] < x) l = m + 1;
        else r = m;
    }
    return l;
}
```

Erklärung:

Gibt die erste Position zurück, an der x eingefügt werden kann (stabile Einfüge-Position).

Binäre Suche – rekursiv

Pseudocode (IHK-Stil):

```
Funktion binarySearchRekursiv(werte: Array<Integer>, x: Integer, links: Integer, rechts: Integer)
    if links > rechts dann
        return -1
    end if
    mitte = (links + rechts) / 2
    if werte[mitte] == x dann
        return mitte
    else if werte[mitte] < x dann
        return binarySearchRekursiv(werte, x, mitte + 1, rechts)
    else
        return binarySearchRekursiv(werte, x, links, mitte - 1)
    end if
end Funktion
```

Java-Beispiel:

```
public static int binarySearchRekursiv(int[] a, int x, int l, int r) {
    if (l > r) return -1;
    int m = (l + r) / 2;
    if (a[m] == x) return m;
    if (a[m] < x) return binarySearchRekursiv(a, x, m + 1, r);
    return binarySearchRekursiv(a, x, l, m - 1);
}
```

Erklärung:

Gleiche Logik, aber rekursiv. Iterativ ist in Prüfungen oft bevorzugt (kein Stackverbrauch).

Lineare Suche – Integer-Array

Pseudocode (IHK-Stil):

```
Funktion lineareSuche(werte: Array<Integer>, x: Integer) → Integer
  for i = 0 to länge(werte) - 1
    if werte[i] == x dann
      return i
    end if
  end for
  return -1
end Funktion
```

Java-Beispiel:

```
public static int lineareSuche(int[] werte, int x) {
  for (int i = 0; i < werte.length; i++) {
    if (werte[i] == x) return i;
  }
  return -1;
}
```

Erklärung:

Robust bei unsortierten Daten. Komplexität $O(n)$. Für kleine n oft ausreichend.

Lineare Suche – Objektliste via equals

Pseudocode (IHK-Stil):

```
Funktion lineareSucheObj(L: Liste<T>, ziel: T) → Integer
  for i = 0 to L.size() - 1
    if L.get(i).equals(ziel) dann
      return i
    end if
  end for
  return -1
end Funktion
```

Java-Beispiel:

```
public static <T> int lineareSucheObj(java.util.List<T> list, T ziel) {
  for (int i = 0; i < list.size(); i++) {
    if (list.get(i).equals(ziel)) return i;
  }
  return -1;
}
```

Erklärung:

Wichtig: equals muss sinnvoll implementiert sein (z. B. ID-basiert).

Lineare Suche – byId (Objekte mit Schlüssel)

Pseudocode (IHK-Stil):

```
Funktion lineareSucheById(L: Liste<Kunde>, id: Integer) → Integer
  for i = 0 to L.size() - 1
    if L.get(i).getId() == id dann
      return i
    end if
  end for
  return -1
end Funktion
```

Java-Beispiel:

```
public static int lineareSucheById(java.util.List<Kunde> list, int id) {
  for (int i = 0; i < list.size(); i++) {
    if (list.get(i).getId() == id) return i;
  }
  return -1;
}
```

Erklärung:

Sehr praxisnah. Variante: direkt das gefundene Objekt zurückgeben statt Index.

Lineare Suche – String-Liste (case-insensitiv)

Pseudocode (IHK-Stil):

```
Funktion findeName(L: Liste<String>, s: String) → Integer
    such = lower(s)
    for i = 0 to L.size() - 1
        if lower(L.get(i)) == such dann
            return i
        end if
    end for
    return -1
end Funktion
```

Java-Beispiel:

```
public static int findeName(java.util.List<String> liste, String s) {
    String such = s.toLowerCase();
    for (int i = 0; i < liste.size(); i++) {
        if (liste.get(i).toLowerCase().equals(such)) return i;
    }
    return -1;
}
```

Erklärung:

Namen oft ohne Beachtung der Groß-/Kleinschreibung vergleichen → beide Seiten `toLowerCase()`.

Lineare Suche – mit Prädikat (Function)

Pseudocode (IHK-Stil):

```
Funktion findeErstes(L: Liste<T>, test: Funktion<T → Boolean>) → Integer
    for i = 0 to L.size() - 1
        if test(L.get(i)) dann
            return i
        end if
    end for
    return -1
end Funktion
```

Java-Beispiel:

```
public static <T> int findeErstes(java.util.List<T> list, java.util.function.Predicate<T> test) {
    for (int i = 0; i < list.size(); i++) {
        if (test.test(list.get(i))) return i;
    }
    return -1;
}
```

Erklärung:

Flexibel für komplexe Kriterien (z. B. Kunde -> Umsatz > 1000).

Lineare Suche – alle Treffer sammeln

Pseudocode (IHK-Stil):

```
Funktion findeAlle(L: Liste<T>, ziel: T) → Liste<Integer>
    ergebnis = neue Liste<Integer>()
    for i = 0 to L.size() - 1
        if L.get(i).equals(ziel) dann
            ergebnis.add(i)
        end if
    end for
    return ergebnis
end Funktion
```

Java-Beispiel:

```
public static <T> java.util.List<Integer> findeAlle(java.util.List<T> list, T ziel) {
    java.util.List<Integer> erg = new java.util.ArrayList<>();
    for (int i = 0; i < list.size(); i++) {
        if (list.get(i).equals(ziel)) erg.add(i);
    }
    return erg;
}
```

Erklärung:

Wenn Mehrfachtreffer möglich sind, gib alle Indizes/Objekte zurück. Praktisch für Auswertungen.